



**JOMO KENYATTA UNIVERSITY
OF
AGRICULTURE & TECHNOLOGY**

SCHOOL OF OPEN, DISTANCE AND eLEARNING

P.O. Box 62000, 00200

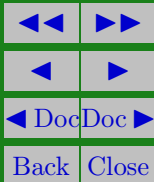
Nairobi, Kenya

E-mail: elearning@jkuat.ac.ke

ICS 2302 SOFTWARE ENGINEERING 1

JKUAT SODEL

©2017





This presentation is intended to be covered within one week. The notes, examples and exercises should be supplemented with a good textbook. Most of the exercises have solutions/answers appearing elsewhere and accessible by clicking the green **Exercise** tag. To move back to the same page click the same tag appearing at the end of the solution/answer.

Errors and omissions in these notes are entirely the responsibility of the author who should only be contacted through the **Department of Curricula & Delivery (SODeL)** and suggested corrections may be e-mailed to elarning@jkuat.ac.ke.



LESSON 4

Software Process Models

Learning Outcomes

A study of this lesson should enable students to:

- Describe the process models
- Describe the principles of Agile Methods



Previously we introduced a simple model of the software life cycle. In practice, one often uses more sophisticated process models. Here, we describe various alternative models to structure the software development process.

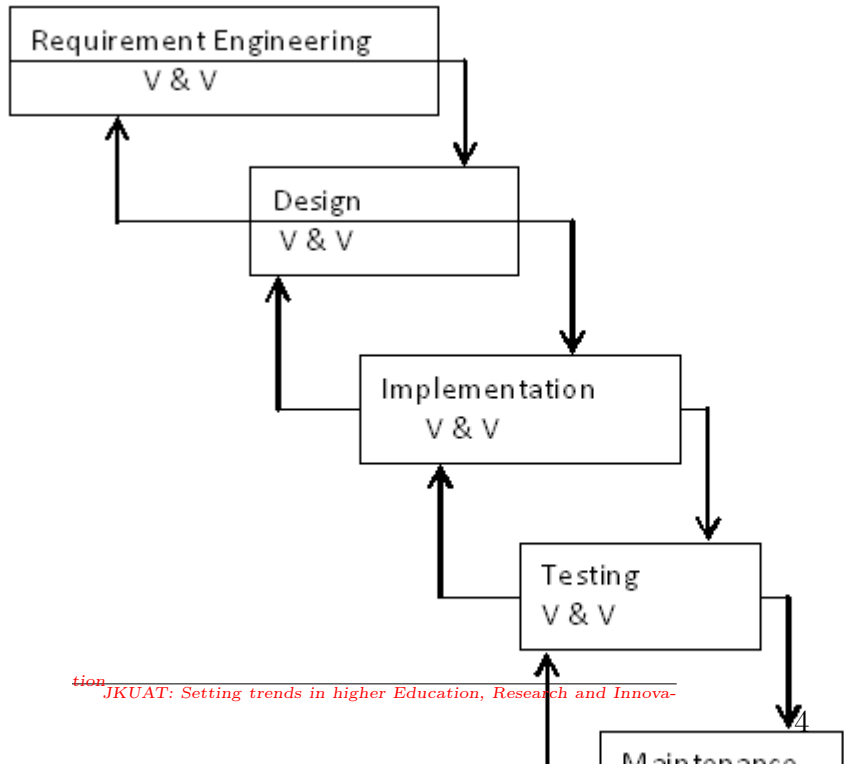
4.1. The Waterfall Model

The waterfall model is essentially a slight variation of the model introduced previously. In the simplest possible classical waterfall model, the different activities in the software life cycle are considered to occur sequentially. The only exception is that two activities that are adjacent are allowed to communicate via feedback. Because of the cascade from one phase to another, this is why this model is known as the 'waterfall model' or software life cycle. Testing software is not an activity which strictly fol-



ICS 2302 Software Engineering 1

lowers the implementation phase. In each phase of the software development process, we have to compare the results obtained against those that are required. In all phases quality has to be assessed and controlled.





In fig 4.1, V & V stands for verification and validation. Verification asks if the system meets its requirements (Are we building the system right?) and thus tries to assess the correctness of the transition to the next phase.

Validation asks if the system meets the user's requirements (Are we building the right system?). Both the previous model and the waterfall model place considerable emphasis on a careful analysis before the system is actually built. We want to prevent putting much energy into constructing a system which later turns out not to satisfy the user's requirements.

We therefore try to identify and tie down the user's requirements as early as possible. These requirements are documented in the requirements specification. On the basis of this docu-

tion
JKUAT: Setting trends in higher Education, Research and Innova-



ICS 2302 Software Engineering 1

ment we may verify in subsequent phases whether or not these requirements are being met.

Since it's difficult in practice, to completely specify the user's requirements, a regular test should also be carried out with the prospective user. These tests are termed validation. Through these validation steps we may prevent the system under development diverging from the, possibly incompletely specified user requirements.

4.2. The Rapid Prototyping Model

In many organizations, technology is advancing so fast that the long lead time between setting requirements and delivering a final tested version of a product seems impractical. Such organizations often follow the rapid prototyping or spiral models of



ICS 2302 Software Engineering 1

software development. These models are iterative and require creation of one or more prototypes as part of the software development process. The software is developed as series of prototypes, with specifications of the individual prototypes changing in response to feedback from users. The rapid prototyping is illustrated in the Figure 4.2.

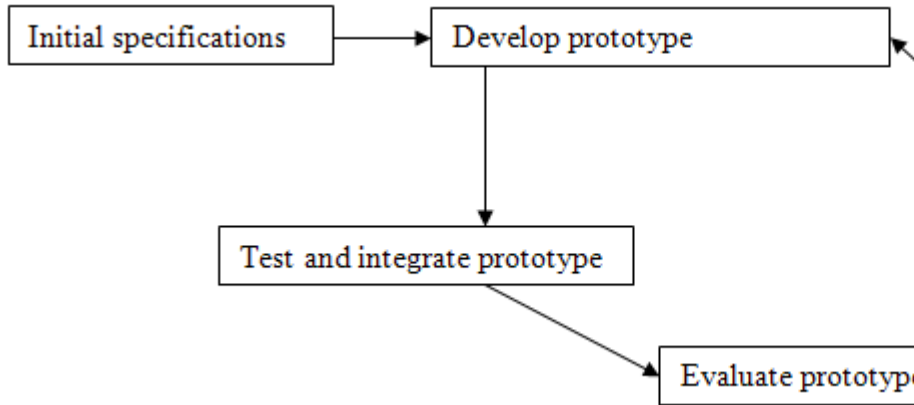


Figure 4.2: Rapid prototyping



The primary assumption of the rapid prototyping model of software development is that the complete requirements specification of the software is not known before the software is designed and implemented. Instead, the software is developed incrementally, with the specification developing along with the software itself. The central point of the rapid prototyping method is that the final software project produced is considered as the most acceptable of a sequence of software prototypes.

4.3. Incremental Development

In preceding section, we discussed a way of using prototypes for which the final system is the last of a series of prototypes. Under careful management control, in order to ensure convergence, the next version is planned to accommodate new or changed user



ICS 2302 Software Engineering 1

requirements. There is another way to work towards the final system in a number of iterations.

We proceed incrementally. The functionality of the system is produced and delivered to the customer in small increments. Starting from the existing situation we proceed towards the desired situation in a number of (small) steps. In each of these steps the phased approach that we know from the waterfall model is employed.

Development of software this way avoids the “Big Bang” effect i.e. for a longtime nothing happens and then suddenly there is a completely new situation. Instead of building the software the software grows. With this incremental approach the user is closely involved in planning the next step. Redirecting the project becomes easier since we may incorporate changed cir-

JKUAT: Setting trends in higher Education, Research and Innovation



cumstances more quickly.

4.4. Rapid Application Development

Rapid Application Development (RAD) has a lot in common with other iterative development process models. It emphasizes user involvement, prototyping, re-use, the use of automated tools and small development teams. In addition to that, it employs the notion of a time box, a fixed time frame within which activities are done. Within RAD, the time frame is decided upon first and then the project tries to realize the requested functionality within that time frame. If it turns out that not all of the functionality can be realized within the time frame, some of the functionality is sacrificed. The agreed deadline however immovable.



The RAD life cycle consists of four phases:

- requirements planning
- user design
- construction
- cutover

4.5. Evolutionary Development

Evolutionary development is based on the idea of developing an initial implementation, exposing this to user and refining this through many versions until an adequate system has been developed (see figure below). Rather than have separate specifications, development and validation activities, these are carried out concurrently with rapid feedback across these activities.

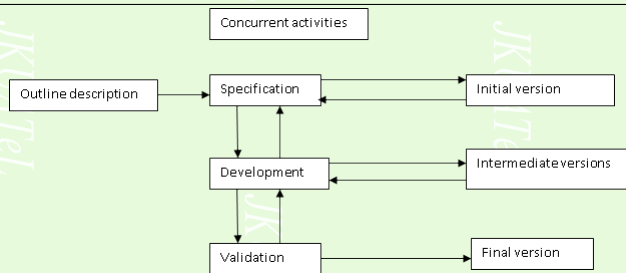


Figure 4.3: Evolutionary development

4.6. The Spiral Model

Another iterative software development lifecycle is the spiral model developed by *Barry Boehm*. The spiral model differs from the rapid prototyping model primarily in the explicit emphasis



ICS 2302 Software Engineering 1

on the assessment of software risk for each prototype during the evaluation period. The term 'risk' is used to describe the potential for disaster in the software project e.g. the software development team is not able to produce the software within the allotted budget and schedule.

In preceding sections we noticed that it is helpful to view software development as a cycle process. Each cycle includes the same phases: requirements engineering, design, implementation and testing.

The phases are not always executed strictly sequentially. During the development of a software system a number of problems have to be solved. Involving a problem, the most difficult parts are often tackled first, or the parts that have the highest risks with respect to a successful completion of the project.



Following this line of thought Boehm suggests a spiral model of the software development process in which each conclusion of the spiral gives rise to the following activities:

- Identifying the sub problem which has the highest associated risk
- Find a solution for that problem.

4.6.1. Boehm's spiral model

- Process is represented as a spiral rather than as a sequence of activities with backtracking.
- Each loop in the spiral represents a phase in the process.
- No fixed phases such as specification or design - loops in the spiral are chosen depending on what is required.



ICS 2302 Software Engineering 1

- Risks are explicitly assessed and resolved throughout the process.

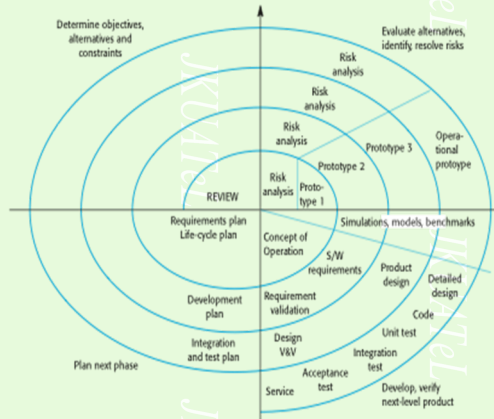


Figure 4.4: Boehm's spiral model of the software process



4.7. The Rational Unified Process

- A modern generic process derived from the work on the UML and associated process.
- Brings together aspects of the 3 generic process models discussed previously.
- Normally described from 3 perspectives
 - A dynamic perspective that shows phases over time
 - A static perspective that shows process activities
 - A practice perspective that suggests good practice.



Phases in the Rational Unified Process

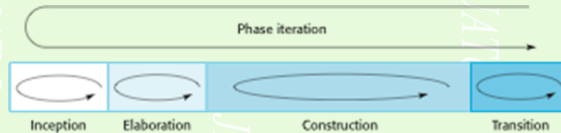


Figure 4.5: RUP Phases

1. Inception - Establish the business case for the system.
2. Elaboration - Develop an understanding of the problem domain and the system architecture.
3. Construction - System design, programming and testing.
4. Transition - Deploy the system in its operating environment.



RUP iteration

In-phase iteration - Each phase is iterative with results developed incrementally.

Cross-phase iteration - As shown by the loop in the RUP model (see fig 4.5), the whole set of phases may be enacted incrementally.

RUP good practice

- Develop software iteratively
 - Plan increments based on customer priorities and deliver highest priority increments first.
- Manage requirements
 - Explicitly document customer requirements and keep track of changes to these requirements.



ICS 2302 Software Engineering 1

- Use component-based architectures
 - Organize the system architecture as a set of reusable components.
- Visually model software
 - Use graphical UML models to present static and dynamic views of the software.
- Verify software quality
 - Ensure that the software meet's organizational quality standards.
- Control changes to software
 - Manage software changes using a change management system and configuration management tools.



ICS 2302 Software Engineering 1

Processes should include activities to cope with change. This may involve a prototyping phase that helps avoid poor decisions on requirements and design.

Processes may be structured for iterative development and delivery so that changes may be made without disrupting the system as a whole.

The Rational Unified Process is a modern generic process model that is organized into phases (inception, elaboration, construction and transition) but separates activities (requirements, analysis and design, etc.) from these phases.

4.8. Agile Software Development

Rapid development and delivery is now often the most important requirement for software systems:



ICS 2302 Software Engineering 1

- Businesses operate in a fast –changing requirement and it is practically impossible to produce a set of stable software requirements
- Software has to evolve quickly to reflect changing business needs.

In rapid software development;

- Specification, design and implementation are inter-leaved
- System is developed as a series of versions with stakeholders involved in version evaluation
- User interfaces are often developed using an IDE and graphical toolset.

Dissatisfaction with the overheads involved in software design methods of the 1980s and 1990s led to the creation of agile meth-



ICS 2302 Software Engineering 1

ods. These methods:

- Focus on the code rather than the design
- Are based on an iterative approach to software development
- Are intended to deliver working software quickly and evolve this quickly to meet changing requirements.

The aim of agile methods is to reduce overheads in the software process (e.g. by limiting documentation) and to be able to respond quickly to changing requirements without excessive rework.



4.8.1. The Principles of Agile Methods

Principle	Description
Customer involvement	Customers should be closely involved throughout the development process. Their role is provide and prioritize new system requirements and to evaluate the iterations of the system.
Incremental delivery	The software is developed in increments with the customer specifying the requirements to be included in each increment.
People not process	The skills of the development team should be recognized and exploited. Team members should be left to develop their own ways of working without prescriptive processes.
Embrace change	Expect the system requirements to change and so design the system to accommodate these changes.
Maintain simplicity	Focus on simplicity in both the software being developed and in the development process. Wherever possible, actively work to eliminate complexity from the system.



4.8.2. Agile Method Applicability

- Product development where a software company is developing a small or medium-sized product for sale.
- Custom system development within an organization, where there is a clear commitment from the customer to become involved in the development process and where there are not a lot of external rules and regulations that affect the software.
- Because of their focus on small, tightly-integrated teams, there are problems in scaling agile methods to large systems.



4.8.3. Problems with Agile Methods

- It can be difficult to keep the interest of customers who are involved in the process.
- Team members may be unsuited to the intense involvement that characterizes agile methods.
- Prioritising changes can be difficult where there are multiple stakeholders.
- Maintaining simplicity requires extra work.
- Contracts may be a problem as with other approaches to iterative development.



Agile methods and software maintenance

Most organizations spend more on maintaining existing software than they do on new software development. So, if agile methods are to be successful, they have to support maintenance as well as original development.

Two key issues:

- Are systems that are developed using an agile approach maintainable, given the emphasis in the development process of minimizing formal documentation?
- Can agile methods be used effectively for evolving a system in response to customer change requests?

Problems may arise if original development team cannot be maintained.



Plan-driven and agile development

Plan-driven development

- A plan-driven approach to software engineering is based around separate development stages with the outputs to be produced at each of these stages planned in advance.
- Not necessarily waterfall model – plan-driven, incremental development is possible
- Iteration occurs within activities.

Agile development

- Specification, design, implementation and testing are interleaved and the outputs from the development process are decided through a process of negotiation during the software development process.



4.8.4. Technical, human, organizational issues in Agile development

Most projects include elements of plan-driven and agile processes. Deciding on the balance depends on:

- Is it important to have a very detailed specification and design before moving to implementation? If so, you probably need to use a plan-driven approach.
- Is an incremental delivery strategy, where you deliver the software to customers and get rapid feedback from them, realistic? If so, consider using agile methods.
- How large is the system that is being developed? Agile methods are most effective when the system can be developed with a small co-located team who can communicate



informally. This may not be possible for large systems that require larger development teams so a plan-driven approach may have to be used.

- What type of system is being developed?
 - Plan-driven approaches may be required for systems that require a lot of analysis before implementation (e.g. real-time system with complex timing requirements).
- What is the expected system lifetime?
 - Long-lifetime systems may require more design documentation to communicate the original intentions of the system developers to the support team.
- What technologies are available to support system development?



ICS 2302 Software Engineering 1

- Agile methods rely on good tools to keep track of an evolving design
- How is the development team organized?
 - If the development team is distributed or if part of the development is being outsourced, then you may need to develop design documents to communicate across the development teams.
- Are there cultural or organizational issues that may affect the system development?
 - Traditional engineering organizations have a culture of plan-based development, as this is the norm in engineering.
- How good are the designers and programmers in the development team?

JKUAT: Setting trends in higher Education, Research and Innovation



- It is sometimes argued that agile methods require higher skill levels than plan-based approaches in which programmers simply translate a detailed design into code.
- Is the system subject to external regulation?
- If a system has to be approved by an external regulator (e.g. the FAA approve software that is critical to the operation of an aircraft) then you will probably be required to produce detailed documentation as part of the system safety case.

4.8.5. Extreme programming

Perhaps the best-known and most widely used agile method.

Extreme Programming (XP) takes an 'extreme' approach to



iterative development.

- New versions may be built several times per day
- Increments are delivered to customers every 2 weeks
- All tests must be run for every build and the build is only accepted if tests run successfully

XP and Agile principles

- Incremental development is supported through small, frequent system releases.
- Customer involvement means full-time customer engagement with the team.
- People not process through pair programming, collective ownership and a process that avoids long working hours.



- Change supported through regular system releases.
- Maintaining simplicity through constant refactoring of code.

4.8.6. Scrum

The Scrum approach is a general agile method but its focus is on managing iterative development rather than specific agile practices.

There are three phases in Scrum.

- The initial phase is an outline planning phase where you establish the general objectives for the project and design the software architecture.
- This is followed by a series of sprint cycles, where each cycle develops an increment of the system.



ICS 2302 Software Engineering 1

- The project closure phase wraps up the project, completes required documentation such as system help frames and user manuals and assesses the lessons learned from the project.

The Scrum process



JKUAT: Setting trends in higher Education, Research and Innovation



The Sprint cycle

- Sprints are fixed length, normally 2–4 weeks. They correspond to the development of a release of the system in XP.
- The starting point for planning is the product backlog, which is the list of work to be done on the project.
- The selection phase involves all of the project team who work with the customer to select the features and functionality to be developed during the sprint.
- Once these are agreed, the team organize themselves to develop the software. During this stage the team is isolated from the customer and the organization, with all communications channeled through the so-called ‘Scrum master’.



ICS 2302 Software Engineering 1

- The role of the Scrum master is to protect the development team from external distractions.
- At the end of the sprint, the work done is reviewed and presented to stakeholders. The next sprint cycle then begins.

Teamwork in Scrum

The 'Scrum master' is a facilitator who arranges daily meetings, tracks the backlog of work to be done, records decisions, measures progress against the backlog and communicates with customers and management outside of the team.

The whole team attends short daily meetings where all team members share information, describe their progress since the last meeting, problems that have arisen and what is planned for the



ICS 2302 Software Engineering 1

following day. This means that everyone on the team knows what is going on and, if problems arise, can re-plan short-term work to cope with them.

Scrum benefits

- The product is broken down into a set of manageable and understandable chunks.
- Unstable requirements do not hold up progress.
- The whole team have visibility of everything and consequently team communication is improved.
- Customers see on-time delivery of increments and gain feedback on how the product works.
- Trust between customers and developers is established and a positive culture is created in which everyone expects the



project to succeed.

4.8.7. Scaling Agile methods

Agile methods have proved to be successful for small and medium sized projects that can be developed by a small co-located team.

It is sometimes argued that the success of these methods comes because of improved communications which is possible when everyone is working together.

Scaling up agile methods involves changing these to cope with larger, longer projects where there are multiple development teams, perhaps working in different locations.



Large systems development

- Large systems are usually collections of separate, communicating systems, where separate teams develop each system. Frequently, these teams are working in different places, sometimes in different time zones.
- Large systems are 'brownfield systems', that is they include and interact with a number of existing systems. Many of the system requirements are concerned with this interaction and so don't really lend themselves to flexibility and incremental development.
- Where several systems are integrated to create a system, a significant fraction of the development is concerned with system configuration rather than original code development

JKUAT: Setting trends in higher Education, Research and Innovation



ICS 2302 Software Engineering 1

- Large systems and their development processes are often constrained by external rules and regulations limiting the way that they can be developed.
- Large systems have a long procurement and development time. It is difficult to maintain coherent teams who know about the system over that period as, inevitably, people move on to other jobs and projects.
- Large systems usually have a diverse set of stakeholders. It is practically impossible to involve all of these different stakeholders in the development process

Scaling out and scaling up

‘Scaling up’ is concerned with using agile methods for developing large software systems that cannot be developed by a small

JKUAT: Setting trends in higher Education, Research and Innovation



team.

‘Scaling out’ is concerned with how agile methods can be introduced across a large organization with many years of software development experience.

When scaling agile methods it is essential to maintain agile fundamentals Flexible planning, frequent system releases, continuous integration, test-driven development and good team communications.

Scaling up to large systems

For large systems development, it is not possible to focus only on the code of the system. You need to do more up-front design and system documentation

Cross-team communication mechanisms have to be designed



ICS 2302 Software Engineering 1

and used. This should involve regular phone and video conferences between team members and frequent, short electronic meetings where teams update each other on progress.

Continuous integration, where the whole system is built every time any developer checks in a change, is practically impossible. However, it is essential to maintain frequent system builds and regular releases of the system.

Scaling out to large companies

Project managers who do not have experience of agile methods may be reluctant to accept the risk of a new approach.

Large organizations often have quality procedures and standards that all projects are expected to follow and, because of their bureaucratic nature, these are likely to be incompatible



ICS 2302 Software Engineering 1

with agile methods.

Agile methods seem to work best when team members have a relatively high skill level. However, within large organizations, there are likely to be a wide range of skills and abilities.

There may be cultural resistance to agile methods, especially in those organizations that have a long history of using conventional systems engineering processes.



Revision Questions

Example  Differentiate between XP and Scrum Agile methods and briefly explain scaling in Agile developments.

Solution:

A particular strength of extreme programming is the development of automated tests before a program feature is created. All tests must successfully execute when an increment is integrated into a system.

The Scrum method is an agile method that provides a project management framework. It is centred round a set of sprints, which are fixed time periods when a system increment is developed.



ICS 2302 Software Engineering 1

Scaling agile methods for large systems is difficult. Large systems need up-front design and some documentation



EXERCISE 1.  Describe the principles of Agile Methods